STUDENT SURVIVAL GUIDE: SAIGONBITE FINAL PROJECT

Step-by-Step Instructions & Deliverable Checklist | Introduction to Software Engineering | Sem01_2627

This guide walks you through the Software Development Life Cycle (SDLC) for your final project. You will work in groups of 2 or 3 to build **SaigonBite**, a cloud-kitchen food delivery platform serving both **Local Customers** and **Foreign Tourists**. Read it together with the Case Study and the **Project Specification**: the specification holds the exact business rules (BR1 to BR18), the order state machine and the grading rubric, and it wins if anything here seems to differ.

> **Dates**
> Part 1 due: **[7:00 Mon 26Oct2026]** | (eLearning & google link)
> Part 2 due: **[7:00 Mon 9Nov2026, end of Week 4]** | (eLeaning & google link)
> Part 3 and Final Submission due: **[7:00 30Nov2026, end of Week 7]** (eLearning & google link)
> **Final Submission: 7Dec2026 (**eLearning & google link with including Part1,2 and 3. **)**

Directory of Visual Guides & Diagrams

To help you succeed, this guide refers to the following diagrams (see your lecture slides for examples):

1. **Requirements Engineering Process:** extract and document FRs and NFRs.
2. **UML Use Case Diagram with Actor Generalization:** base Customer versus Local Customer and Foreign Tourist.
3. **CRC Cards:** object-oriented planning before the class diagram.
4. **UML State Diagram:** the life of an order from Placed to Delivered, Cancelled or Failed.
5. **Strategy Design Pattern:** one payment interface with one class per payment method.
6. **UI Design Progression:** the 4 stages of Figma design you must submit.
7. **Git Workflow:** how your team collaborates on code.
8. **Equivalence Partitioning and Boundary Value Analysis:** the techniques for your QA tests.
9. **Requirements Traceability Matrix (RTM):** maps your final code and tests to your initial requirements.

The 7 Weeks at a Glance

Week	Focus	What must exist by the end of the week
1	Analysis	Team, methodology, use case ownership, first draft of FRs with IDs
2	Analysis (Part 1)	Proposal and Requirements document, Ambiguity Log, Presentation 1
3	Design	Use case diagram, use case specifications, CRC cards, Order state diagram

Week	Focus	What must exist by the end of the week
4	Design (Part 2)	Class diagram/ERD, architecture diagram, Figma prototype, Presentation 2
5	Build and checkpoint	Running skeleton shown to the instructor; Change Request issued
6	Build and test	Use cases complete, pricing rules coded, EP/BVA tests running
7	Finish (Part 3 and Final)	RTM, Change Request Impact Note, final PDF, video, live demo

PART 1: Project Proposal & Requirements Engineering

(Due: [26Oct2026]) Total Value: 20% of Final Grade

What you need to do

1. **Form Your Team & Methodology:** Decide roles for your 2-3 members. Choose your SDLC methodology (for example Agile Scrum) and justify why it works for a 7-week project.
2. **Assign 1 Core Use Case Per Student:** Choose main workflows that include complete CRUD (Create, Read, Update, Delete). The specification lists candidate use cases A to F. **Someone must take "Place and Manage Orders"**, which owns the shared pricing and payment module.
   - Example (team of 3): Student A owns "Manage Menu & Brands (CRUD)". Student B owns "Manage Hubs & Capacity (CRUD)". Student C owns "Place and Manage Orders (CRUD) and the pricing and payment module".
   - Example (team of 2): Student C above plus one more use case such as "Manage Vouchers" or "Manage Deliveries".
   - Do not forget: the team must also assign ownership of the **2 System Reports** (Revenue and Hub Utilization).
3. **Identify the Actors:** Customer (base), Local Customer, Foreign Tourist, Kitchen Staff, Shipper and System Admin.
4. **Write Requirements (FRs and NFRs):**
   - Write Functional Requirements ("The system shall…") with **IDs (FR1, FR2…)** for your use cases and reports. Cite the business rules they depend on (BR1…BR18).
   - Define at least 3 measurable Quality Attributes (NFR1…). Mandatory baselines: performance (price calculation under 1.5 seconds for up to 30 lines), maintainability (a new payment method means adding one class), usability (responsive, English/Vietnamese) and security (hashed passwords, role-based access). Add **at least two of your own**.
5. **Write an Ambiguity Log:** The specification deliberately leaves some details open. Find at least 3 gaps (for example who assigns the shipper, or what is refunded after a failed delivery), write down the assumption you made, and ask the instructor on Canvas if the answer matters.

Deliverable 1: Proposal & Requirements Document (PDF)

Your document must contain:

- Title Page (project name, team members, student IDs, tier selection).
- Project Proposal (roles, methodology, use case ownership table).
- Problem Statement.
- Functional Requirements (FRs) with IDs.
- Quality Attributes (NFRs) with IDs.
- Ambiguity Log.

Presentation 1: The Solution Pitch (5-7 mins)

- Prepare 3-5 slides. Every member must speak to explain the problem and present the specific use case they own.

PART 2: Software Design, Architecture & UI/UX

(Due: [9Nov2026]) Total Value: 25% of Final Grade

What you need to do

1. **Map User Interactions:** Draw an overall UML Use Case diagram showing actor generalization (Customer to Local Customer and Foreign Tourist).
2. **Deep Dive into Your Assigned Use Case:** Write a formal Use Case Specification (pre-conditions, main flow, alternate flows, post-conditions) citing FR and BR IDs, and draw a Sequence Diagram or Activity Diagram.
3. **Model the Order Lifecycle:** As a team, draw a UML **State Diagram** with the 8 order states and the transitions in the specification, including who may trigger each one.
4. **Object & Quality Design:**
   - Create CRC Cards to plan how your objects interact before drawing your final classes.
   - Draw a formal UML Class Diagram or ERD. It must show the **Strategy pattern** for payments and how Local Customers and Tourists share one schema.
   - Draw your Architecture Diagram (3-Tier or MVC). Explain how your design uses quality principles (SOLID, separation of concerns, high cohesion and low coupling) to meet your NFRs, and say **where the pricing rules live and why**.
5. **Design the UI in Figma:** Create an interactive prototype that follows the 4 stages (sketch to prototype).
   - Web tiers: use responsive Bootstrap grid layouts.
   - Show: the 2 reports, the menu and brand browsing for a hub, a checkout screen with a price breakdown (subtotal, discounts, delivery fee, surcharges, total) and payment buttons for Cash, MoMo, VNPay, Apple Pay and PayPal, and an order tracking screen.

Deliverable 2: System Design Document (PDF)

Your document must contain:

- Overall Use Case Diagram.
- Detailed specifications and Sequence/Activity diagrams for each student's use case.
- The Order State Diagram.
- CRC Cards and Class Diagram/ERD.
- The Architecture Diagram and Quality Design explanation.
- Figma deliverable: a public link to your Figma workspace.

Presentation 2: The Design Review (7-10 mins)

- Project your CRC cards and class diagram and explain your database logic and the Strategy pattern.
- Run your Figma prototype live for the class. Expect an individual question about your own use case.

PART 3 & FINAL SUBMISSION

(Due: [30Nov2026]) Part 3 Value: 35% | Final Report & Video Value: 20%

What you need to do

1. **Week 5 Checkpoint:** Show the instructor a running skeleton (database schema plus one use case working end to end). This is not graded, but a team that cannot show one will be asked to meet the instructor. The **Change Request** is issued this week.
2. **Code Your Use Cases:** Implement the logic exactly as you designed it.
   - Implement the pricing and payment rules **BR1 to BR14** exactly as written: minimum order, delivery fee tiers, rush-hour and rain surcharges, free delivery, the Quiet Kitchen discount, vouchers, calculation order, rounding and the cash limit.
   - Enforce the order state machine in the **business layer** (not just in the UI).
   - Use Bootstrap for styling if you choose a web tier.
   - **Every** team member must push code to GitHub (more than 10 meaningful commits each, mostly on their own use case).
3. **Absorb the Change Request:** Implement it with small, local changes. Do not edit the existing payment classes if the change is a new payment method. Write a **one-page Change Request Impact Note** listing classes added and modified.
4. **Detailed Design Justification:** Write a short section explaining what changed between your Part 2 diagrams and your actual code. Did you add a database table? Did a controller get split? Explain why.
5. **Perform QA Testing:**
   - Create a Test Plan table (Test ID, description, input data, expected result, actual result, pass/fail, and the FR and BR IDs covered).

  - Write and run unit tests on the pricing logic using **Equivalence Partitioning and Boundary Value Analysis** for every numeric threshold. You need **at least 20** such test cases.
  - Add **at least 5 tests** that try invalid order-state transitions and check that they are rejected.
  - (Tier 4 only) Write automated tests with Selenium or Playwright.
6. **Verification (RTM):** Create a Requirements Traceability Matrix linking each FR and NFR to its use case, design element, code component and test cases, with a verdict. Start it in Part 1 and keep it up to date, so the final version is not a last-minute scramble.
7. **Compile the Final Report:** Merge Parts 1, 2 and 3 into a single PDF with bookmarks. Add a Lessons Learnt section where each student reflects on their experience, challenges and how they overcame them. Apply the feedback you received after each presentation.
8. **Record a Video Demo:** Record your screen showing the software working end to end: one order from a Local Customer, one from a Foreign Tourist, a state change by kitchen staff and shipper, and both reports.
9. **Declare Your AI Use:** Complete the AI usage log table in the specification (tool, purpose, prompt summary, what you kept or changed, how you verified it) and keep your full prompts and outputs ready in case the instructor asks.

Deliverables to Upload

1. **Source Code:** A **tagged release** in your GitHub repository (this is the official copy). A .zip of the project folder is a welcome backup.
2. **Contribution Statement:** One page, signed by all members, saying who owns which use case and components.
3. **Final Comprehensive PDF Report (10%):** Bookmarked, containing all SDLC phases, the design justification, QA test plans, the RTM, the Change Request Impact Note and Lessons Learnt.
4. **Video Demo Product (10%):** An .mp4 file or YouTube link showing your use cases and reports working end to end.
5. **Appendix.pdf:** The Generative AI Usage Log.
6. **Peer review form:** Submitted individually and confidentially at the end of Week 7.

Presentation 3: Final Product Demo (10-15 mins)

- Live-demo the software to the class. Students take turns running their own use cases (with complete CRUD) and the 2 reports.
- Open your GitHub history to prove everyone contributed. Each student answers at least one individual question about their own work.

Survival Tips

- **Write the pricing rules as code first.** A small pure function that takes (subtotal, hub load, distance, time, rain flag, voucher) and returns the price is easy to test and easy to change. Keep it out of controllers and views.

- **Test the boundaries, not just the middle.** Most bugs hide at 29% versus 30%, 12:59 versus 13:00, and 299,999 versus 300,000.
- **Follow the calculation order (BR10) literally.** Rounding only at the end is the most common mistake.
- **Never skip the state machine.** If the UI hides a button but the business layer allows the transition, it is a bug.
- **Start the RTM in Week 2 and ask early.** Add a row for each requirement as you write it. If a rule is unclear, post it on Canvas and record the answer in your Ambiguity Log.
- **Commit small and often.** Ten meaningful commits means ten reasons, not ten file uploads at 2 a.m. on the last night.
- **Use AI as an assistant, not an author.** You must be able to explain every diagram and line of code in the live Q&A.
- **Pick a tier you can finish, and rehearse the demo.** A working Tier 2 project beats an unfinished Tier 3 one. Rehearse with a clean database and keep a backup recording.